

Forecast hourly ride demand across Addis Ababa

A ride-hailing company needs to know how many trips riders will request in each of 12 zones, for every hour of 1–14 November 2025 — so drivers can be in the right place at the right time.

Goal: Clean → analyze → model → ship a working ops demo.



One clock, one grain: zone × hour

Three raw tables to combine

- **Trip history:** Hourly trips per zone (Jan–Oct 2025) — the target we learn from
- **Weather (hourly):** Temp, rain, humidity, wind — rain lifts demand, but clocks must match
- **Events calendar:** Holidays, matches, concerts, closures — local spikes around venues

The clock problem

- Trip hours: already EAT (UTC+3)
- Weather ...Z timestamps: UTC → convert +3h
- Proof: temperature peaks at 15:00 after conversion, not 12:00
- Mis-aligning by 3–5 hours kills the rain–demand signal.

Join audit

- Train rows before → after joins: 84,614 → 84,614 (no duplication)
- Weather match rate: ~89% (gaps filled with ffill/bfill)
- Confirmed events matched ≥1 zone-hour: 151
- Unmatched / cancelled excluded: 8 unmatched · 6 cancelled

Demand is growing — and rain and events are not the same everywhere

Demand is growing — November needs a trend

**~44.8k →
~65.9k**

Weekly city trips:
~44.8k → ~65.9k
(+47% Jan→Oct)

A January-level
baseline would
under-predict
November

- Keep: trend_day + recent lags (≥ 14 days back)

Rain and events are real — but not everywhere the same

Rainy vs dry lifts
transport/residential ($\sim \times 1.3$) but
cuts Merkato (market) to $\times 0.83$

Holidays: most reduce demand
(Genna $\times 0.72$); Adwa holds
flat/up

Football: post-match 2h
uplift $\times 2.85 > \text{pre } \times 2.17$
> during $\times 1.44$

So what: calendar-only
models miss the biggest
operational surprises.

Beat seasonality, then prove it over time

Baselines (18–31 Oct)

Mean predictor	RMSE 29.47; MAE 20.89
Seasonal naive (zone × dow × hour)	RMSE 15.36; MAE 7.44

Candidates (same split)

Ridge	RMSE 14.93; weak on interactions
LightGBM	RMSE 13.69; fast to tune
HistGradientBoosting	RMSE 13.61; best single split

Ablation (rolling RMSE)

Calendar + zone + trend	16.43
+ weather	15.87 (−0.56)
+ events	15.53 (−0.90)
+ both	14.96 (−1.48, −9%)

Final model: LightGBM +
weather + events + 14-day-safe
lags (chosen for
speed/tuneability; accuracy
within noise of HistGB).

Rolling-origin (4 fortnights)

- LightGBM: 14.96 ± 2.16
RMSE
- Seasonal naive: 16.77 ± 1.39
- Wins 4/4 folds

Hygiene: chronological splits
only · no ops leakage · weather
+ event features required ·
nothing fitted on test

Where we miss — and what we shipped

Error analysis

- Error scales with volume: Merkato RMSE highest; MAE ~19–23% of mean demand across zones
- Top errors: rows with ~10× trips vs drivers → suspected corrupted targets (not real demand)
- Fix: drop suspect targets in training → clean rolling RMSE 10.07 → 9.60

Final validation score

- RMSE: 9.60 clean rows; 14.72 all rows
- MAE: 6.20 clean rows; 6.54 all rows

In plain language: typically off by ~6 trips/zone-hour (~19%) ≈ ~5 drivers (at 1.3 trips/driver-hour).

Demo

Live: <https://addis-ride-demand-forecasting-mjc8skkfdzkgtw6nstthdp.streamlit.app/>

Local: `streamlit run app/app.py`

Pick a zone + date (1–14 Nov) → hourly forecast, peak hour, drivers needed, expected fares, weather/events looked up automatically.

What we'd try next

1. Zone-specific holiday effects (not one binary flag)
2. Stronger handling of unlisted spikes / monitoring for drift
3. Quantile forecasts for driver buffer planning (80% band already in demo)

